

DSA0502 – Query Processing for Data Science

Class Activity

Course Faculty: Dr. B .SHYAMALA GOWRI

Reg. No.: 192324164

Student Name: Sweety Roselin A

Date: 30/07/2026

1. Missing Value Imputation

Question

Write a Python program to perform Missing Value Imputation using the mean method.

Aim

To replace missing values with the mean of the available values.

Code

```
import pandas as pd
```

```
marks = [75, 82, None, 91, 68, None, 84]
```

```
df = pd.DataFrame({  
    "Student": ['A','B','C','D','E','F','G'],  
    "Marks": marks  
})
```

```
print("Original Dataset")
```

```
print(df)
```

```
mean_marks = df["Marks"].mean()
```

```
print("\nMean =", mean_marks)
```

```
df["Marks"].fillna(mean_marks, inplace=True)
```

```
print("\nAfter Imputation")
```

```
print(df)
```

```
print("\nNew Class Average =", df["Marks"].mean())
```

Output

```
... Original Dataset
      Student Marks
0          A  75.0
1          B  82.0
2          C   NaN
3          D  91.0
4          E  68.0
5          F   NaN
6          G  84.0

Mean = 80.0

After Imputation
      Student Marks
0          A  75.0
1          B  82.0
2          C  80.0
3          D  91.0
4          E  68.0
5          F  80.0
6          G  84.0

New Class Average = 80.0
```

Result

Thus, missing values were successfully replaced using the mean.

2. Z-Score Outlier Detection

Question

Write a Python program to detect an outlier using the Z-score.

Aim

To identify whether a value is an outlier using the Z-score method.

Code

```
mean = 61.22
```

```
std = 42.63
```

```
x = 180
```

```
z = (x - mean) / std
```

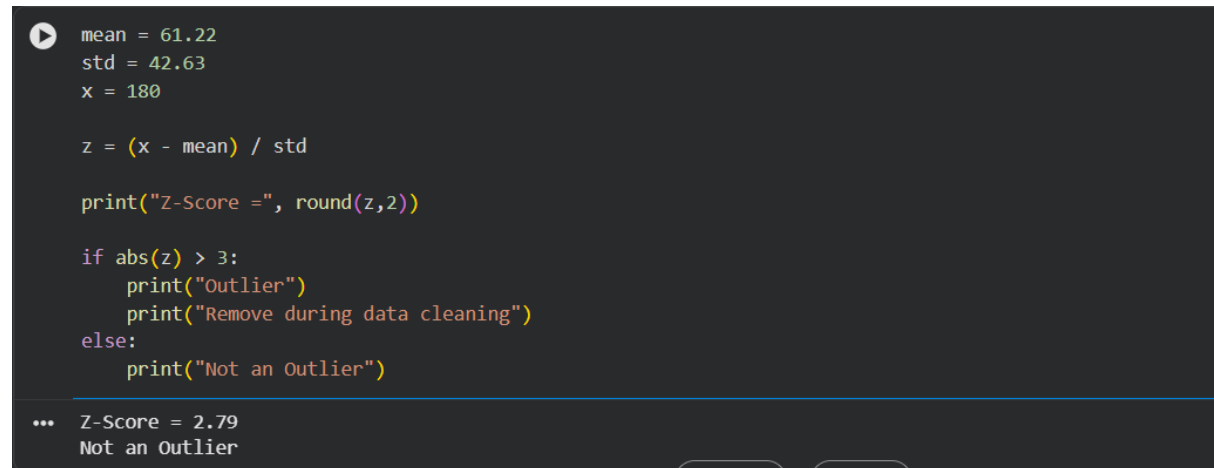
```
print("Z-Score =", round(z,2))
```

```
if abs(z) > 3:
```

```
    print("Outlier")
```

```
    print("Remove during data cleaning")
else:
    print("Not an Outlier")
```

Output



```
mean = 61.22
std = 42.63
x = 180

z = (x - mean) / std

print("Z-Score =", round(z,2))

if abs(z) > 3:
    print("Outlier")
    print("Remove during data cleaning")
else:
    print("Not an Outlier")

... Z-Score = 2.79
    Not an Outlier
```

Result

Thus, the Z-score indicates that **180 is not an outlier** since $|Z| < 3$.

3. Data Normalization

Question

Write a Python program to perform Min-Max Normalization.

Aim

To normalize salary values into the range [0,1].

Code

```
salaries = [42000,56000,70000,85000,98000]
```

```
minimum = min(salaries)
```

```
maximum = max(salaries)
```

```
print("Normalized Values")
```

```
for s in salaries:
```

```
    norm = (s-minimum)/(maximum-minimum)
```

```
    print(s,"->",round(norm,2))
```

```

salary = 70000

value = (salary-minimum)/(maximum-minimum)

print("\nNormalized value of 70000 =",round(value,2))

print("\nMinimum =",0)
print("Maximum =",1)

```

Output

```

... Normalized Values
42000 -> 0.0
56000 -> 0.25
70000 -> 0.5
85000 -> 0.77
98000 -> 1.0

Normalized value of 70000 = 0.5

Minimum = 0
Maximum = 1

```

Result

Thus, Min-Max Normalization was performed successfully.

4. Standardization

Question

Write a Python program to standardize a student's score.

Aim

To calculate the Z-score of a student's marks.

Code

```

score = 82
mean = 70
std = 8

z = (score-mean)/std

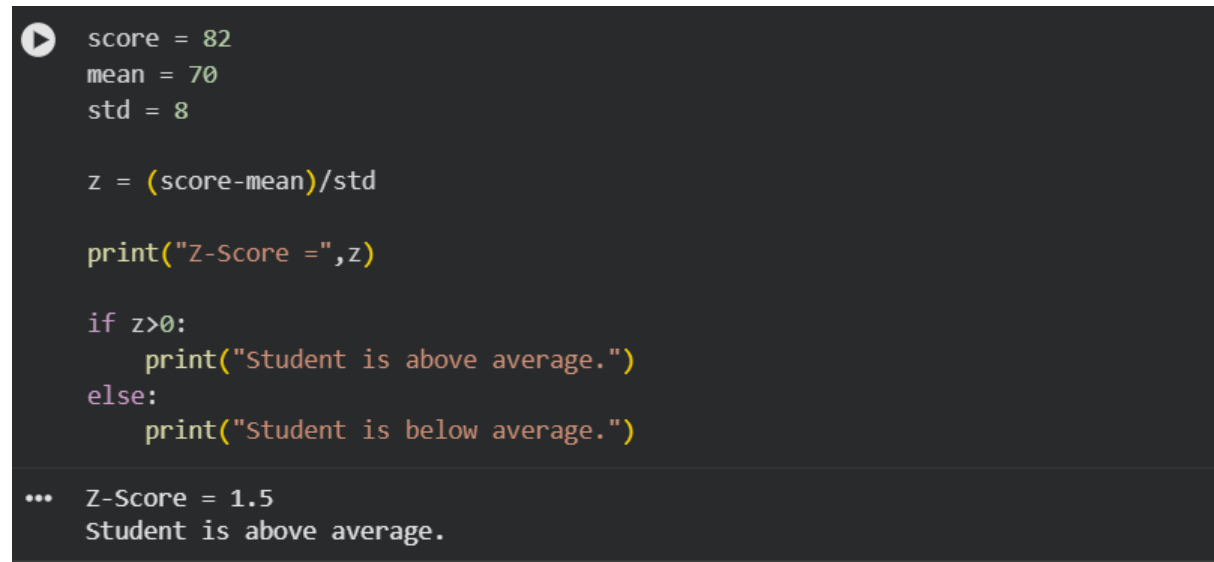
print("Z-Score =",z)

if z>0:

```

```
    print("Student is above average.")
else:
    print("Student is below average.")
```

Output



```
▶ score = 82
  mean = 70
  std = 8

  z = (score-mean)/std

  print("Z-Score =",z)

  if z>0:
      print("Student is above average.")
  else:
      print("Student is below average.")

... Z-Score = 1.5
     Student is above average.
```

Result

Thus, the standardized score was calculated successfully.

5. SQL Numerical Problem

Question

Write a Python program to calculate total sales and average selling price.

Aim

To calculate total revenue and average price.

Code

```
import pandas as pd

df = pd.DataFrame({
    "Product":["Laptop","Mouse","Keyboard","Monitor"],
    "Quantity":[8,25,15,10],
    "Price":[45000,650,1200,14000]
})

df["Revenue"] = df["Quantity"]*df["Price"]
```

```
print(df)
```

```
print("\nTotal Sales =",df["Revenue"].sum())
```

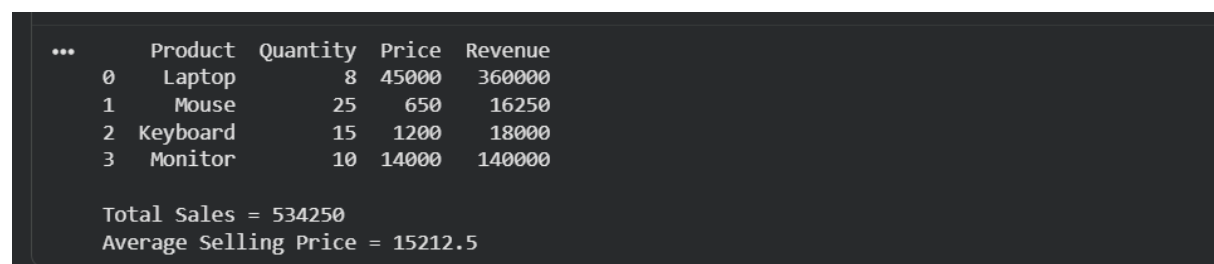
```
print("Average Selling Price =",df["Price"].mean())
```

SQL Query

```
SELECT SUM(Quantity*Price) AS TotalRevenue
```

```
FROM Sales;
```

Output



```
...      Product  Quantity  Price  Revenue
0      Laptop        8    45000   360000
1       Mouse       25     650    16250
2   Keyboard       15    1200    18000
3    Monitor       10   14000   140000

Total Sales = 534250
Average Selling Price = 15212.5
```

Result

Thus, the total revenue and average selling price were calculated successfully.

6. Correlation Calculation

Question

Write a Python program to calculate Pearson Correlation.

Aim

To find the relationship between X and Y.

Code

```
import pandas as pd
```

```
x=[2,4,6,8,10]
```

```
y=[3,5,7,9,11]
```

```
df=pd.DataFrame({"X":x,"Y":y})
```

```
corr=df["X"].corr(df["Y"])
```

```
print("Correlation =",corr)
```

```
if corr>0:
```

```
    print("Positive Relationship")
```

Output

```
Correlation = 0.9999999999999999
Positive Relationship
```

Result

Thus, Pearson's correlation coefficient was calculated successfully.

7. Grouping and Aggregation

Question

Write a Python program to perform grouping and aggregation.

Aim

To calculate total and average sales region-wise.

Code

```
import pandas as pd
```

```
df=pd.DataFrame({
    "Region":["East","West","East","South","West","East"],
    "Sales":[250,320,280,300,350,270]
})
```

```
print("Total Sales")
```

```
print(df.groupby("Region")["Sales"].sum())
```

```
print("\nAverage Sales")
```

```
print(df.groupby("Region")["Sales"].mean())
```

```
print("\nHighest Total Sales Region")
```

```
print(df.groupby("Region")["Sales"].sum().idxmax())
```

Output

```
Total Sales
Region
East      800
South     300
West       670
Name: Sales, dtype: int64

Average Sales
Region
East      266.666667
South     300.000000
West      335.000000
Name: Sales, dtype: float64

Highest Total Sales Region
East
```

Result

Thus, grouping and aggregation were performed successfully.

8. Histogram Frequency Calculation

Question

Write a Python program to draw a histogram.

Aim

To calculate frequency and draw a histogram.

Code

```
import matplotlib.pyplot as plt

marks=[25,30,34,36,41,45,48,52,55,57,60,63,65,67,72,75,78,82,86,90]

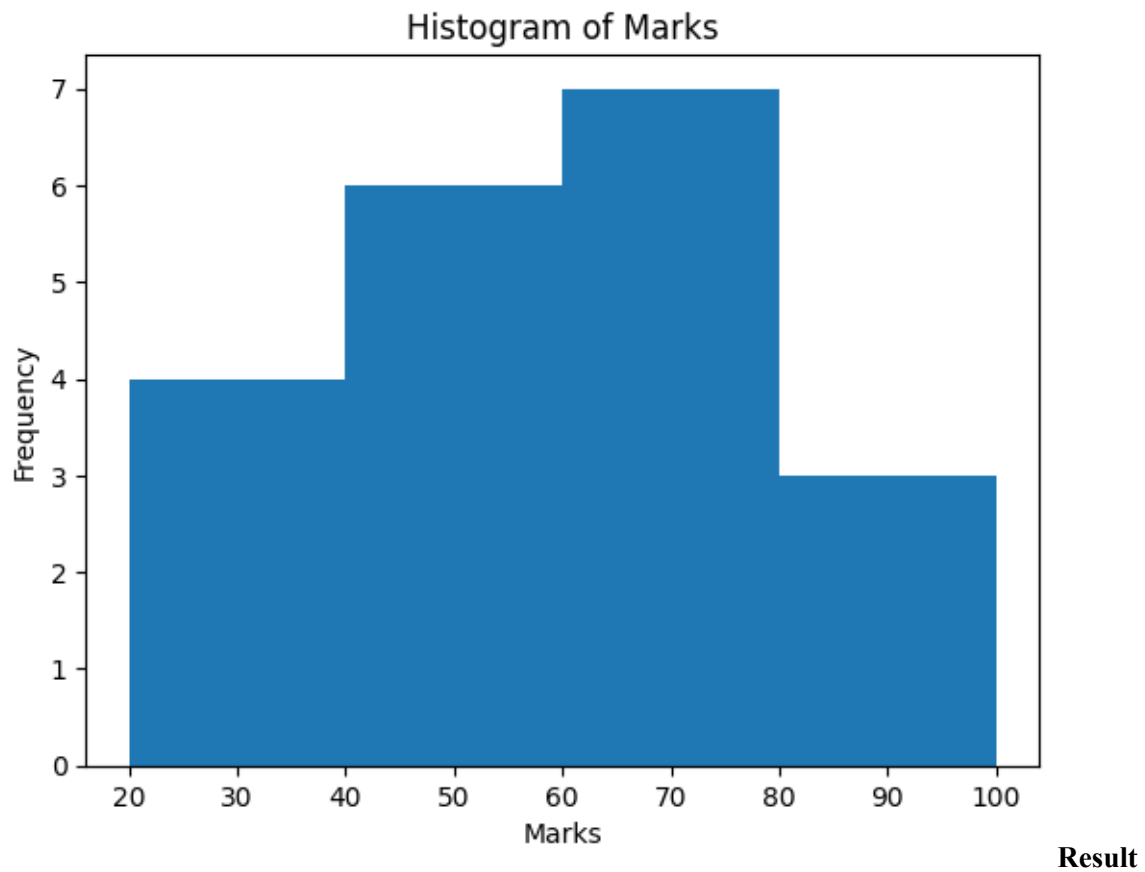
bins=[20,40,60,80,100]

plt.hist(marks,bins=bins)

plt.title("Histogram of Marks")
plt.xlabel("Marks")
plt.ylabel("Frequency")

plt.show()
```

Output



Thus, the histogram was plotted successfully.

9. Data Visualization Analysis

Question

Write a Python program to plot monthly sales using a line chart.

Aim

To analyze monthly sales and growth.

Code

```
import matplotlib.pyplot as plt
```

```
months=["Jan","Feb","Mar","Apr","May","Jun"]
```

```
sales=[45,52,49,61,70,65]
```

```
growth=[]
```

```
for i in range(1,len(sales)):
```

```
    growth.append(sales[i]-sales[i-1])

print("Month-to-Month Growth")

for i in range(len(growth)):
    print(months[i+1],growth[i])

print("\nMaximum Increase =",max(growth))

plt.plot(months,sales,marker='o')

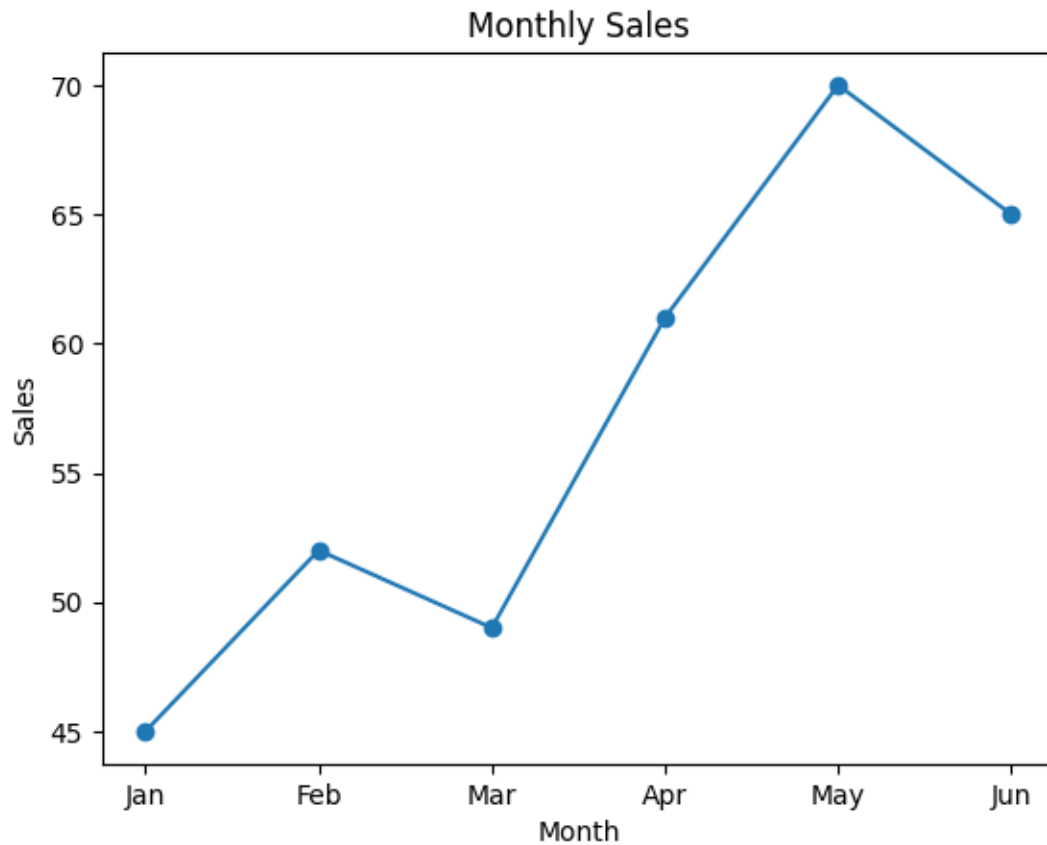
plt.title("Monthly Sales")
plt.xlabel("Month")
plt.ylabel("Sales")

plt.show()
```

Output

```
... Month-to-Month Growth
    Feb  7
    Mar -3
    Apr 12
    May  9
    Jun -5

Maximum Increase = 12
```



Result

Thus, the monthly sales were analyzed and plotted successfully.

10. Duplicate Detection

Question

Write a Python program to calculate duplicate and missing record percentages.

Aim

To determine the percentage of records requiring data cleaning.

Code

```
total=2500
```

```
duplicate=180
```

```
missing=70
```

```
unique=total-duplicate
```

```
duplicate_percent=(duplicate/total)*100
```

```
missing_percent=(missing/total)*100
```

```
cleaning=((duplicate+missing)/total)*100
```

```
print("Unique Records =",unique)
```

```
print("Duplicate Percentage =",round(duplicate_percent,2),"%")
```

```
print("Missing Percentage =",round(missing_percent,2),"%")
```

```
print("Cleaning Required =",round(cleaning,2),"%")
```

Output

```
Unique Records = 2320  
Duplicate Percentage = 7.2 %  
Missing Percentage = 2.8 %  
Cleaning Required = 10.0 %
```

Result

Thus, the duplicate records, missing records, and total percentage of data requiring cleaning were calculated successfully.